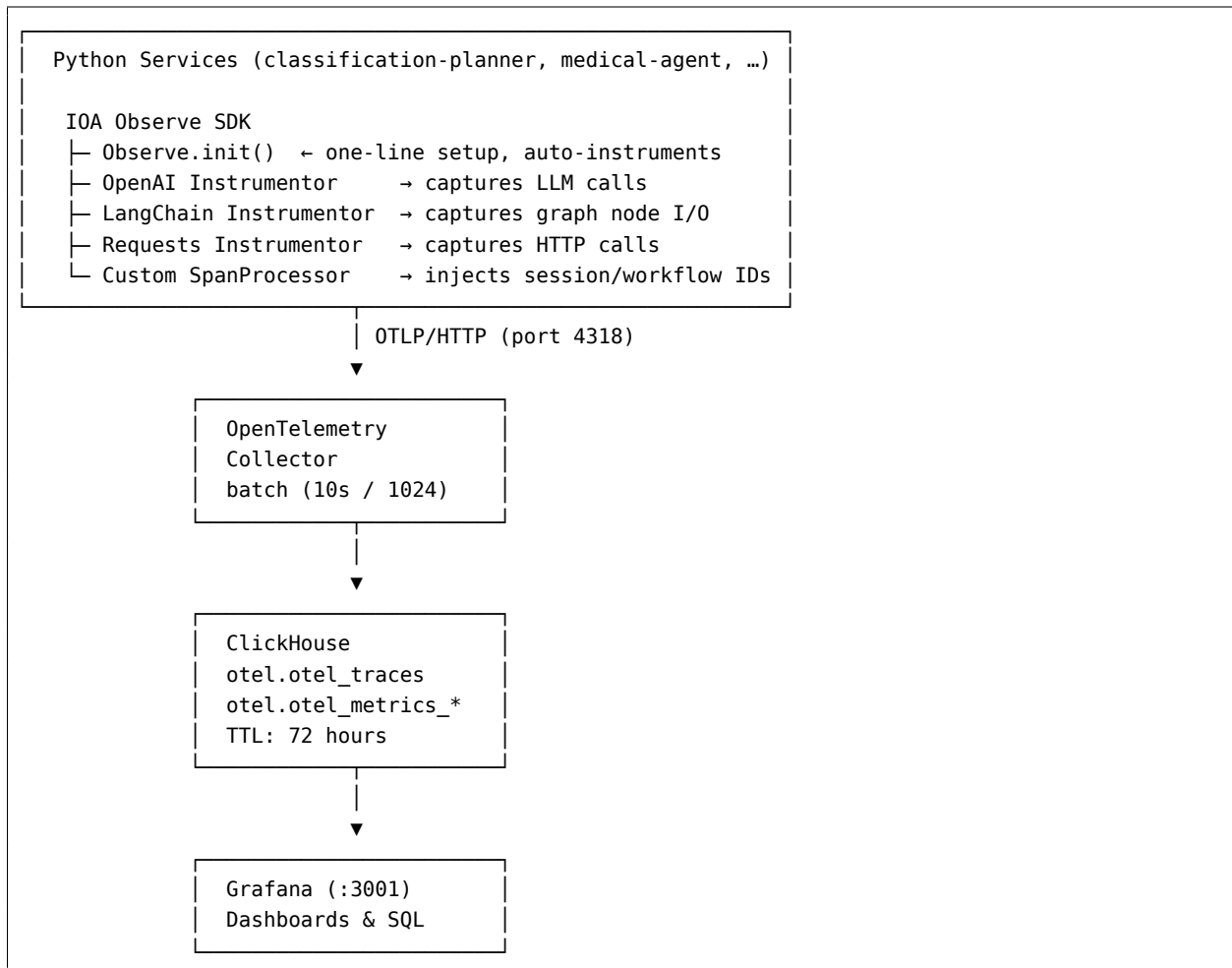


Observability for Multi-Agent System

2026-02-19

1 Observability Architecture

1.1 Components



1.2 How It Works

1. **IOA Observe SDK** is embedded in every Python service via `Observe.init()`. It automatically detects installed packages (OpenAI, LangChain, requests) and activates corresponding

instrumentors.

- Each instrumentor wraps API calls with OpenTelemetry **spans**. When an LLM call is made, the instrumentor intercepts the request and response, recording system prompts, completions, token usage, model parameters, and timing.
- A custom **SpanProcessor** enriches every span with session IDs, workflow names, and LangGraph node associations (e.g., which node — `supervisor`, `reason`, `act_classify` — produced this LLM call).
- Spans are exported via OTLP/HTTP to the **OTel Collector**, which batches them and writes to **ClickHouse** for persistent storage and SQL querying.

2 What the Observability Component Captures

2.1 Per-LLM-Call Data (Span Attributes)

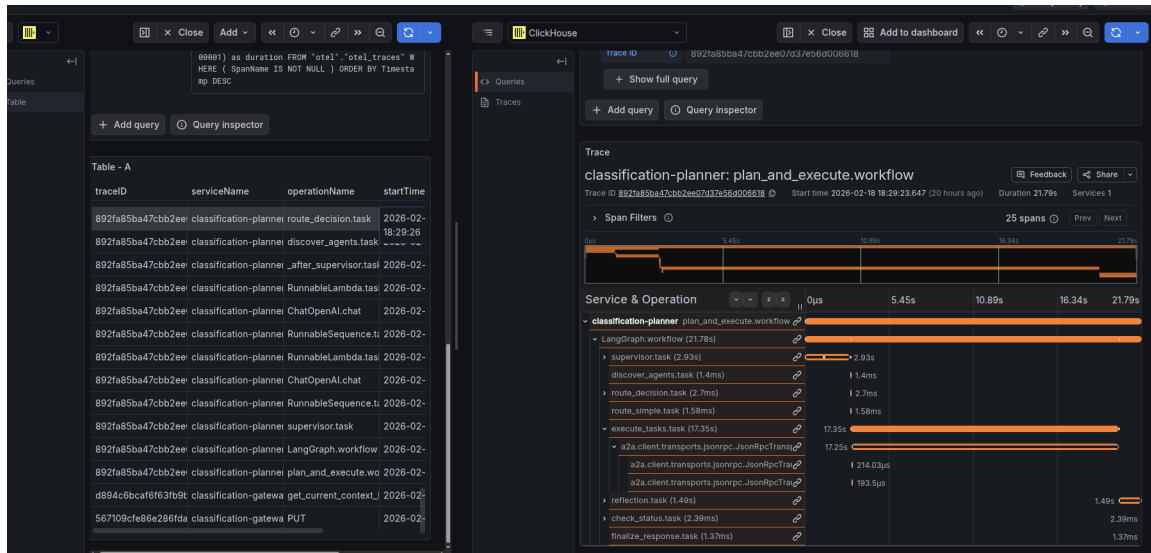


Figure 1: Span

Every LLM invocation produces a span with these attributes:

Attribute	Description	Example
<code>gen_ai.request.model</code>	Requested model	<code>gpt-5.2</code>
<code>gen_ai.response.model</code>	Actual model that responded	<code>gpt-5.2-2025-12-11</code>
<code>gen_ai.prompt.0.content</code>	Full system prompt	<code>"You are an intent guard..."</code>
<code>gen_ai.prompt.1.content</code>	Full user prompt	<code>"Classify this medical image..."</code>
<code>gen_ai.completion.0.content</code>	Full LLM response	<code>{"is_classification": true, ...}</code>

Attribute	Description	Example
<code>gen_ai.usage.input_tokens</code>	Prompt token count	264
<code>gen_ai.usage.output_tokens</code>	Completion token count	39
<code>gen_ai.request.temperature</code>	Sampling temperature	0.0
<code>gen_ai.request.max_tokens</code>	Max output tokens	300

2.2 Per-Node Data (LangGraph Workflow)

Attribute	Description	Example
<code>ioa_observe.association.properties.langgraph_node</code>	Which graph node	supervisor, reason, act_classify
<code>ioa_observe.association.properties.langgraph_step</code>	Step number	1, 2, 3
<code>ioa_observe.entity.input</code>	Node input (JSON)	<code>{"messages": [...], "image_data": ...}</code>
<code>ioa_observe.entity.output</code>	Node output (JSON)	<code>{"classification": "fracture", ...}</code>
<code>ioa_observe.entity.path</code>	Execution path	<code>LangGraph.workflow.supervisor</code>

2.3 Cross-Service Communication

Attribute	Description	Example
TraceId	Distributed trace ID	892fa85ba47cbb2ee07d37e56d006618
ParentSpanId	Parent span (enables tree reconstruction)	Links planner → medical-agent
A2A transport spans	JSON-RPC message send/receive	<code>JsonRpcTransport.send_message</code> (17,250ms)
Agent handoff links	<code>link.type=agent_handoff</code> , <code>link.from_agent</code>	Proves causality chain

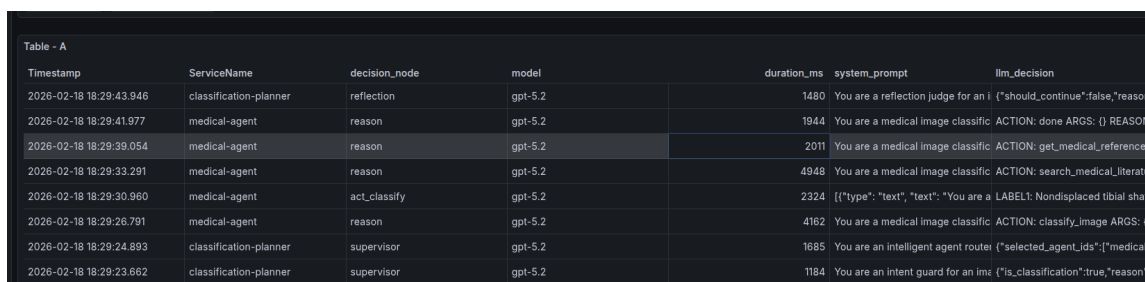
2.4 Timing and Performance

Field	Description	Example
Timestamp	Span start (nanosecond precision)	2026-02-19 00:29:23.662018337
Duration	Span duration (nanoseconds)	1184000000 (= 1,184ms)
StatusCode	Execution result	Ok, Error, Unset

2.5 Metrics

Metric	Description
gen_ai.client.ioa.agent.execution_success_rate	Agent execution success rate
gen_ai.client.ioa.agent.connection_reliability	Inter-agent connection success rate
gen_ai.client.ioa.agent.error_recovery_rate	Self-recovery percentage
number_active_agents	Currently active agent count
llm_call_count	Total LLM invocations

2.6 Record-Level View — What Is Actually Captured Per LLM Decision



Timestamp	ServiceName	decision_node	model	duration_ms	system_prompt	llm_decision
2026-02-18 18:29:43.946	classification-planner	reflection	gpt-5.2	1480	You are a reflection judge for an i ("should_continue":false,"reason"	
2026-02-18 18:29:41.977	medical-agent	reason	gpt-5.2	1944	You are a medical image classific ACTION: done ARGS: {} REASON:	
2026-02-18 18:29:39.054	medical-agent	reason	gpt-5.2	2011	You are a medical image classific ACTION: get_medical_reference /	
2026-02-18 18:29:33.291	medical-agent	reason	gpt-5.2	4948	You are a medical image classific ACTION: search_medical_literatur	
2026-02-18 18:29:30.960	medical-agent	act_classify	gpt-5.2	2324	[{"type": "text", "text": "You are a LABEL: Nondisplaced tibial shaft	
2026-02-18 18:29:26.791	medical-agent	reason	gpt-5.2	4162	You are a medical image classific ACTION: classify_image ARGS: {'	
2026-02-18 18:29:24.893	classification-planner	supervisor	gpt-5.2	1685	You are an intelligent agent router ("selected_agent_ids":["medical-i	
2026-02-18 18:29:23.662	classification-planner	supervisor	gpt-5.2	1184	You are an intent guard for an ima ("is_classification":true,"reason":	

Figure 2: Record

Sections 2.1–2.5 describe span attributes at the technical level. From a **decision record** perspective, each LLM call in the workflow produces a record containing three key pieces of captured content: the **system prompt** (instructions given to the LLM), the **LLM decision** (what the model actually output), and **execution metadata** (who, when, how long, which model). The following table summarizes what the observability component captured for each of the 8 LLM decision points in one complete classification request:

Record	Service	Decision Node	System Prompt (captured)	LLM Decision (captured)
1	classification-planner	supervisor (Intent Guard)	Full intent guard instructions: accept/reject rules for classification requests	{"is_classification": true, "reason": "...image analysis/classification"}
2	classification-planner	supervisor (Agent Router)	Full agent catalog (3 agents with IDs, descriptions, skills, tags) + selection instructions	{"selected_agent_ids": ["medical-image-classifier---", "confidence": 0.93}
3	medical-agent	reason (ReAct #1)	Full ReAct prompt: 3 available tools, strategy rules, output format	ACTION: classify_image + args with request_id and S3 image ref
4	medical-agent	act_classify (Vision)	Classification prompt + task JSON (image ref, constraints, confidence threshold)	LABEL1: Nondisplaced tibial shaft fracture (0.86), LABEL2: Fibular shaft fracture (0.79), LABEL3: Nutrient artery canal (0.34)

Record	Service	Decision Node	System Prompt (captured)	LLM Decision (captured)
5	medical-agent	reason (ReAct #2)	Same ReAct prompt (with updated conversation history)	ACTION: search_medical_literature + PubMed query args
6	medical-agent	reason (ReAct #3)	Same ReAct prompt (with updated conversation history)	ACTION: get_medical_reference + condition="tibia fracture"
7	medical-agent	reason (ReAct #4)	Same ReAct prompt (with updated conversation history)	ACTION: done + summary of classification and evidence gathered
8	classification-planner	reflection	Full reflection judge prompt: STOP/CONTINUE rules, mismatch detection criteria	{"should_continue": false, "reason": "confidence 0.86 >= 0.75 threshold", "mismatch_detected": false}

Each record also captures execution metadata:

Record	Model	Duration	Input Tokens	Output Tokens
1	gpt-5.2	1,184ms	264	39
2	gpt-5.2	1,685ms	530	74
3	gpt-5.2	4,162ms	647	249
4	gpt-5.2	2,324ms	480	81
5	gpt-5.2	4,948ms	665	236
6	gpt-5.2	2,011ms	751	73
7	gpt-5.2	1,944ms	822	64
8	gpt-5.2	1,480ms	501	57

This means the observability component captures the **complete decision chain** — for every LLM call, we know:

- **What instructions were given** (full system prompt text)
- **What the LLM decided** (full completion text)
- **Which model made the decision** (model name and version)
- **How long it took** (millisecond-level timing)
- **How much it cost** (exact token counts)
- **Where in the workflow** (service name + LangGraph node)

The SQL query used to extract these records:

```
SELECT
  Timestamp,
  ServiceName,
  SpanAttributes['ioa_observe.association.properties.langgraph_node'] AS decision_node,
  SpanAttributes['gen_ai.request.model'] AS model,
  Duration/1000000 AS duration_ms,
  SpanAttributes['gen_ai.prompt.0.content'] AS system_prompt,
```

```

SpanAttributes['gen_ai.completion.0.content'] AS llm_decision
FROM otel.otel_traces
WHERE SpanName IN ('ChatOpenAI.chat', 'openai.chat')
ORDER BY Timestamp DESC

```

3 Can Observability Data Be Used for Attestation?

3.1 What the Current Observability Data CAN Prove

The captured trace data provides strong **behavioral evidence**:

Attestation Dimension	Evidence Available	Captured Data
Who acted	Which service and LangGraph node	ServiceName, langgraph_node
What was decided	Full LLM input/output	gen_ai.prompt.*.content, gen_ai.completion.*.content
Which model	Requested and actual model	gen_ai.request.model, gen_ai.response.model
When it happened	Nanosecond timestamps	Timestamp, Duration
In what order	Parent-child span tree + timestamps	TraceId, SpanId, ParentSpanId
With what parameters	Temperature, max_tokens, etc.	gen_ai.request.*
How much it cost	Token counts per call	gen_ai.usage.input_tokens, output_tokens
Was it successful	Status and reflection judgment	StatusCode, reflection output
Cross-service causality	A2A transport spans + agent handoff links	link.type=agent_handoff, link.from_agent
Workflow compliance	Step sequence matches expected graph	langgraph_step, span tree structure

3.2 What the Current System CANNOT Prove

Gap	Description	What's Needed
Tamper-proofing	Trace data in ClickHouse can be modified by anyone with DB access	Cryptographic signing of spans before export
Non-repudiation	A service could claim it didn't produce a span	Digital signatures tied to service identity (SPIFFE SVID)
Completeness	No guarantee all spans were delivered (network loss, SDK crash)	Acknowledged delivery + span sequence numbers
Model integrity	Proves which model name was called, not that the model wasn't swapped at the API level	Model fingerprinting / response attestation from LLM provider

Gap	Description	What's Needed
Data provenance chain	Spans are independent records; no hash chain linking them	Merkle tree or blockchain-style chaining of span hashes
Third-party verifiability	Only the ClickHouse operator can query traces	Signed trace exports with verifiable timestamps

3.3 Bridging Observability to Attestation — How

The current observability system provides the **data layer** for attestation. To build a complete attestation system, it needs to be combined with other AGNTCY components:

```

What's needed for full attestation: _____|

Layer 1: Observability |IOA Observe SDK|
"Was the data tampered with?" |←NOT YET INTEGRATED|
Full trace of every decision || _____||

Layer 2: Integrity |Directory Sign API (Sigstore)|
"Who is this agent?" |←NOT YET INTEGRATED|
Cryptographic signing of ||
trace records / span digests || _____||

Layer 3: Identity |Identity Service (W3C VC Badge)|
"Was the agent authorized?" |←NOT YET INTEGRATED|
Access control verification ||
against captured actions | _____|

```